DOSSIER DE PROJET · IA GÉNÉRATIVE

Recherche sémantique dans mes notes

Démonstration Python / Streamlit avec embeddings, index vectoriel, évaluation et traçabilité des prompts

Module	Semantic Search
Stack	Python 3, sentence-transformers, FAISS / NumPy, Streamlit, pytest
Étudiant	_____
Professeur	_____
Date	26 octobre 2026
Version	1.0 — dossier Word

But de ce document
Présenter le projet, les outils d'IA utilisés, les prompts qui ont permis de le construire et les preuves de vérification. Le fichier est conçu pour être ouvert et présenté dans Microsoft Word.

Sommaire

1. Présentation du projet
2. Ce qui a été livré
3. Fonctionnement du moteur
4. Choix techniques
5. Outils d'IA utilisés
6. Démarche et prompts de construction
7. Discussion et prompts avec Claude.ai
8. Prompt prêt à l'emploi pour construire la démo
9. Résultats et vérifications
10. Limites et points d'honnêteté
11. Lancer la démonstration
12. Conclusion

Annexe A — Arborescence du projet

Annexe B — Questions probables du professeur

Annexe C — Déclaration d'usage de l'IA

À lire avant l'oral
Le projet est fonctionnel, mais il faut savoir expliquer le pipeline, les métriques et la différence entre un modèle d'embeddings, un index vectoriel et un LLM de génération. Les questions de l'annexe B préparent cet échange.

1. Présentation du projet

Ce projet construit un moteur de recherche sémantique sur des notes de cours. La question de départ est simple : comment retrouver un passage pertinent quand l'étudiant ne se souvient pas des mots exacts ? Une recherche par mots-clés échoue souvent sur les synonymes, les paraphrases ou les reformulations.

Exemple : la question « comment éviter que le modèle apprenne par cœur ? » doit pouvoir retrouver une note qui parle de sur-apprentissage, de régularisation ou de capacité de généralisation, même si ces mots ne sont pas répétés dans la question.

Objectif et périmètre

- Retrouver les passages les plus proches du sens d'une question.
- Classer les résultats par similarité décroissante.
- Afficher le titre, l'extrait et le score de chaque passage.
- Comparer le moteur sémantique à une baseline par mots-clés.
- Fournir une démonstration Streamlit simple et vérifiable.
- Rester 100 % local : aucune clé API et aucun service cloud obligatoire.
- Documenter les prompts, les choix et les vérifications pour l'oral.

Le module est autonome dans le projet, sous ai-study-assistant/modules/semantic_search/. Le template PHP existant n'a pas été modifié. La démonstration est entièrement Python + Streamlit.

Ce que le projet ne prétend pas être

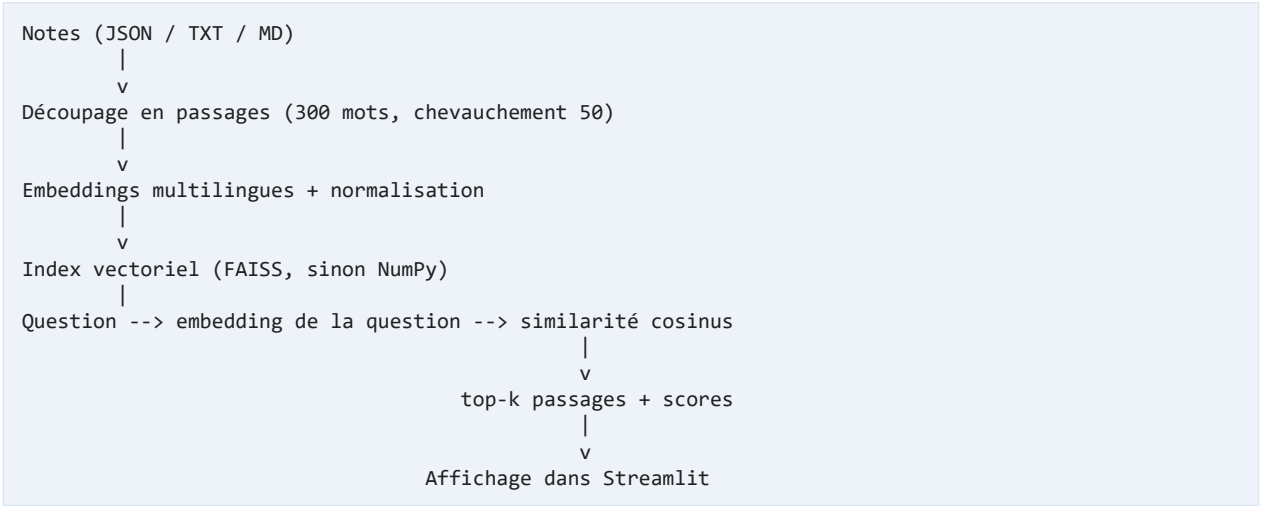
Le corpus de démonstration est petit : 6 notes. Les résultats mesurés ne prouvent pas une supériorité générale du sémantique sur les mots-clés. Ils montrent que le pipeline fonctionne et que les métriques sont calculées réellement, sans résultats inventés.

2. Ce qui a été livré

Élément	État	Détail
Moteur sémantique	Réalisé	src/module.py : validation, découpage, embeddings, recherche.
Découpage	Réalisé	300 mots et 50 mots de chevauchement dans la démo ; 80/15 par défaut dans le moteur.
Embeddings	Réalisé	paraphrase-multilingual-MiniLM-L12-v2, exécuté dans le .venv.
Index vectoriel	Réalisé	FAISS prévu ; repli NumPy sur cette machine car la DLL FAISS est bloquée.
Persistance	Réalisé	index.json, embeddings.npy et faiss.index si disponible.
Recherche	Réalisé	Similarité cosinus, top-k, titres, extraits et scores.
Évaluation	Réalisé	Precision@k, Recall@k, MRR et baseline mots-clés.
Interface Streamlit	Réalisé	Notes d'exemple, import, recherche, compteurs et évaluation.
Tests	Réalisé	27 tests pytest passent avec un embedder déterministe.
RAG Ollama	Optionnel	Client et tests présents ; contrôles retirés de l'interface simplifiée.
Template PHP	Non modifié	Aucun branchement PHP nécessaire.

Le dépôt contient aussi une documentation technique dans search/README.md, une carte de rôle dans search/role-card.md et un journal de prompts dans prompts.md. Le rapport Word synthétise ces éléments pour le professeur.

3. Fonctionnement du moteur



Étapes réelles

1. Charger les notes depuis un fichier JSON, TXT ou Markdown.
2. Découper chaque note en passages en essayant de respecter les phrases.
3. Encoder chaque passage avec sentence-transformers.
4. Normaliser les vecteurs et construire l'index (FAISS ou NumPy).
5. Encoder la question de l'utilisateur avec le même modèle.
6. Calculer la similarité entre la question et chaque passage.
7. Trier et retourner les top-k passages avec titre, extrait et score.
8. Optionnellement, transmettre les extraits à un LLM local pour une réponse RAG.

Rôle des différents modèles

• sentence-transformers n'est pas un LLM génératif : il transforme un texte en vecteur.
• FAISS n'est pas un modèle : c'est une bibliothèque d'index vectoriel.
• Ollama est un LLM local optionnel : il génère une réponse à partir des passages retrouvés.
• Le moteur de recherche fonctionne sans Ollama ; le LLM ne remplace jamais la recherche.

Point important pour l'oral
La recherche ne demande pas à un LLM de « lire toutes les notes ». Elle encode les passages une fois, puis compare des vecteurs. C'est plus rapide et plus contrôlable. Le LLM intervient seulement après, si on veut rédiger une réponse.

4. Choix techniques

Choix	Justification
Modèle multilingue MiniLM	Adapté au français, léger, 384 dimensions, bon compromis vitesse / qualité.
300 mots / 50 de chevauchement	Assez de contexte pour une idée entière, sans index trop fragmenté. Fixé dans la démo pour des résultats reproductibles.
Similarité cosinus	Comparaison directe de vecteurs normalisés ; score interprétable.
FAISS puis NumPy	FAISS accélère la recherche ; NumPy assure le fonctionnement même si FAISS ne se charge pas.
Streamlit	Démo rapide, lisible par le professeur, sans front-end à maintenir.

Choix	Justification
pytest	Tests reproductibles sans télécharger le vrai modèle.
JSON / TXT / MD	Formats simples à lire et à expliquer ; PDF/DOCX non inclus dans le MVP.
Index persistant	Évite de recalculer les embeddings et rend le pipeline plus réaliste.

Le moteur garde des paramètres réglables dans le code, mais la démo les fixe. Ce choix évite d'exposer trop de contrôles à l'utilisateur et rend la démonstration plus claire pour l'oral.

5. Outils d'IA utilisés

Le projet a été produit dans un dialogue avec Claude.ai. L'IA propose le code, les tests et les corrections ; l'étudiant les copie dans le projet, exécute les commandes et valide les résultats. C'est ce qui permet de construire et vérifier la démo sans saisir tout le code à la main.

Outil / modèle	Type	Rôle dans le projet	Statut
Claude.ai	Assistant conversationnel	Rédaction du code, des tests, des prompts et de la documentation.	Utilisé
sentence-transformers	Modèle d'embeddings	Transformation des passages et des questions en vecteurs.	Utilisé à l'exécution
FAISS	Bibliothèque d'index	Recherche vectorielle rapide.	Prévu ; repli NumPy ici
Ollama + llama3.2	LLM local	Réponse RAG optionnelle à partir des extraits.	Code + faux serveur ; modèle réel non lancé
ChatGPT	Assistant conversationnel	Reformulation, préparation de l'oral, revue.	Non nécessaire à l'exécution

Quel LLM choisir ?

- Pour écrire et modifier le projet : Claude.ai en mode conversation. On lui donne le contexte et les erreurs ; il propose le code, que l'étudiant copie et teste.
- Pour expliquer ou reformuler : ChatGPT ou Claude en mode conversation suffisent.
- Pour une réponse RAG locale : Ollama avec un modèle comme llama3.2. C'est optionnel et indépendant de la recherche.
- Pour la recherche : sentence-transformers, pas un LLM génératif. Le modèle d'embedding est l'élément central.

Transparence
Claude.ai a été utilisé pour produire le code et la documentation par échanges de prompts. L'étudiant a copié les fichiers, lancé les tests et validé les mesures. Aucune exécution n'est attribuée à l'IA.

6. Démarche et prompts de construction

Le travail a été découpé en étapes. À chaque étape, un prompt décrit le résultat attendu, les contraintes et les vérifications. L'IA produit, l'utilisateur relit, lance les tests et demande une correction si nécessaire. Cinq étapes suffisent : cadrage, moteur, évaluation, interface, vérification. Les prompts ci-dessous sont représentatifs du dialogue réel ; les formulations ont parfois été nettoyées pour la lisibilité.

Prompt 1 — Cadrage

```
Tu es mon assistant de développement pour un projet de moteur de recherche
sémantique sur des notes de cours.

Stack imposée : Python 3, sentence-transformers, FAISS, Streamlit, pytest.

Crée uniquement la structure suivante, sans écrire la logique de recherche :
search/README.md, requirements.txt, prompts.md, data/, src/module.py,
src/evaluate.py, tests/test_search.py, demo_app.py.

Contraintes :
- contrats clairs, docstrings et TODO ;
- tests skipped tant que la logique n'existe pas ;
- documenter l'objectif, le pipeline, les limites et l'évaluation ;
- rester honnête sur ce qui n'est pas encore fait.
```

Prompt 2 — Implémentation du moteur

```
Implémente src/module.py en respectant les contrats existants.

Contraintes :
- charger les notes depuis data/ ;
- découper en passages avec chevauchement ;
- utiliser un modèle sentence-transformers multilingue ;
- normaliser les vecteurs et utiliser FAISS ;
- retourner des SearchResult triés par score décroissant ;
- gérer corpus vide, requête vide et top_k invalide ;
- expliquer chaque choix et signaler les limites ;
- ne pas modifier les tests pour les faire passer artificiellement.
```

Prompt 3 — Tests et évaluation

```
Écris les tests de tests/test_search.py et implémente src/evaluate.py.

Contraintes :
- cas nominal, corpus vide, requête vide, top_k invalide ;
- vérifier le tri, la taille de sortie et les champs ;
- créer un petit jeu de requêtes annotées ;
- calculer Recall@k, Precision@k et MRR ;
- comparer à une baseline par mots-clés ;
- les tests doivent échouer si le contrat n'est pas respecté.
```

Prompt 4 — Interface Streamlit

```
Implémente demo_app.py avec Streamlit.

Contraintes :
- chargement des notes et indexation avec mise en cache ;
- champ de question et nombre de résultats ;
- affichage du titre, de l'extrait et du score ;
- message explicite si aucune note n'est chargée ;
- aucun appel réseau obligatoire ;
- ne pas masquer les erreurs du moteur.
```

Prompt 5 — Index persistant

```
Ajoute une étape de création explicite des embeddings et de l'index.

Contraintes :
- réutiliser le chargement et le découpage déjà présents dans src ;
- sauvegarder passages, modèle et vecteurs ;
- recharger l'index sans recalculer les embeddings ;
- utiliser FAISS si disponible, sinon NumPy ;
- ajouter une commande CLI ;
- tester sauvegarde, rechargement et fichiers invalides ;
- ne pas inventer de vecteurs.
```

Prompt 6 — Simplification et correction de la démo

```
Simplifie demo_app.py :
- supprime le choix du modèle, les sliders de taille de passage et de
  chevauchement, ainsi que la case notes d'exemple ;
- fixe le modèle à paraphrase-multilingual-MiniLM-L12-v2 ;
- fixe 300 mots par passage et 50 mots de chevauchement ;
- garde le titre « Recherche sémantique dans mes notes » ;
- supprime le sous-titre « Module 2... » ;
- ne montre plus le backend ni le nom technique du modèle dans l'interface.

Puis corrige : si aucun fichier n'est importé, la démo ne doit pas disparaître.
Charge les notes d'exemple par défaut, garde la barre de recherche, le slider
du nombre de résultats, les compteurs Notes / Passages indexés et l'évaluation.
```

Prompt 7 — Vérification finale

```
Vérifie le projet sans modifier les résultats pour les rendre plus beaux.

- lance compileall et pytest ;
- teste l'interface Streamlit sans fichier puis avec un fichier ;
- lance une recherche réelle et une évaluation ;
- note les erreurs, les limites et les dépendances réellement utilisées ;
- mets à jour la documentation si un chiffre ou un comportement change.
```

7. Discussion et prompts avec Claude.ai

Les échanges ci-dessous montrent le pilotage par prompts. Ils sont reformulés pour être lisibles, mais le sens et l'ordre des décisions sont conservés.

Étudiant	Réponse / production de Claude.ai
Construis le module de recherche sémantique, sans écrire la logique au départ.	Proposition de l'arborescence, des contrats, des tests skipped et de la documentation de cadrage.
Implémente le moteur, les tests et l'évaluation.	Code du moteur avec découpage, embeddings, FAISS/NumPy, métriques et baseline mots-clés.
Ajoute une démo Streamlit et un LLM local optionnel avec Ollama.	Code de l'interface, du client Ollama, du faux serveur de test et de la gestion des erreurs.
Ajoute la création des embeddings et l'index persistant.	Code de la CLI build_index, de index.json et de embeddings.npy, avec sauvegarde et rechargement.
Simplifie la démo : garde seulement l'import, le slider et la recherche.	Code simplifié : contrôles supprimés, modèle et découpage fixés, sous-titre retiré.

Pourquoi tout disparaît ? Garde la recherche, l'évaluation, les notes et les passages.	Correction proposée : chargement automatique des notes d'exemple, compteurs et évaluation restaurés.
Enlève aussi l'affichage du backend et du modèle technique.	Code corrigé : mentions retirées de l'interface, réglages conservés dans le code.

8. Prompt prêt à l'emploi pour construire la démo

Le prompt suivant peut être donné à un assistant conversationnel (Claude.ai, ChatGPT, etc.) pour reconstruire une démo équivalente sans écrire tout le code à la main. Il insiste sur les étapes, les tests et l'honnêteté des résultats.

```
Tu es mon binôme de développement Python. Je te fournis les fichiers et les
messages d'erreur ; tu proposes le code et les tests, sans prétendre exécuter.

Objectif : construire une démo Streamlit de recherche sémantique dans des notes.

Stack : Python 3, sentence-transformers, FAISS si disponible, Streamlit, pytest.

Contraintes :
1. Travaille par petites étapes et explique ce que tu vas modifier.
2. Ne fabrique aucun résultat ni aucune métrique.
3. Si les dépendances lourdes manquent, utilise un embedder de test pour pytest.
4. Découpe les notes en passages de 300 mots avec 50 mots de chevauchement.
5. Utilise le modèle paraphrase-multilingual-MiniLM-L12-v2.
6. Affiche titre, extrait et score ; garde la recherche, les compteurs et
   l'évaluation sur les notes d'exemple.
7. Ne montre pas le backend ni le nom technique du modèle dans l'interface.
8. Après chaque étape, donne-moi les commandes compileall et pytest à lancer ;
   je te transmets les erreurs pour correction.
9. Termine par un compte rendu : fichiers, commandes, tests, limites.

Commence par me demander les fichiers nécessaires, puis propose un plan court.
```

Prompt de vérification

```
Agis comme un relecteur critique. Vérifie :
- les affirmations non démontrées ;
- les résultats inventés ou préremplis ;
- les tests qui ne testent rien ;
- les dépendances réellement installées ;
- les différences entre ce que la démo affiche et ce que le README promet.

Ne réécris pas tout : liste les problèmes et les commandes de vérification.
```

Prompt pour préparer l'oral

```
Explique-moi ce projet comme à un professeur, en 5 minutes :
1. le problème ;
2. le pipeline embeddings -> index -> requête -> top-k ;
3. le rôle de FAISS et le repli NumPy ;
4. les métriques Precision@k, Recall@k et MRR ;
5. la part de l'IA dans la production et ce que j'ai vérifié moi-même.
Pose-moi ensuite 5 questions pièges et corrige mes réponses.
```

9. Résultats et vérifications

Vérification	Résultat
Tests pytest	27 passed
Compilation Python	compileall OK
Démarrage Streamlit	HTTP 200
Index réel	6 notes -> 6 passages (300/50)
Recherche réelle	5 résultats, meilleur score 0.683
Interface sans fichier	notes d'exemple, recherche et évaluation visibles
Import d'un fichier	index reconstruit, notes d'exemple remplacées
FAISS	DLL bloquée sur cette machine ; repli NumPy fonctionnel
Ollama	client testé avec un faux serveur ; modèle réel non lancé

Évaluation mesurée

Réglages	Moteur	Precision	Recall	MRR
CLI, 80/15, k=3	Sémantique	0.333	0.833	0.722
CLI, 80/15, k=3	Mots-clés	0.389	1.000	0.806
Démo, 300/50, k=5	Sémantique	0.233	1.000	0.889
Démo, 300/50, k=5	Mots-clés	0.233	1.000	0.639

Ces chiffres sont mesurés, pas inventés. Sur ce corpus minuscule, la baseline lexicale reste forte et le sémantique n'est pas meilleur partout. Le MRR de la démo est meilleur, mais cela ne suffit pas à conclure à une supériorité générale. C'est une limite à assumer clairement à l'oral.

10. Limites et points d'honnêteté

- Le corpus de démonstration contient seulement 6 notes : les métriques sont fragiles.
- La recherche sémantique n'est pas toujours meilleure que les mots-clés sur un petit corpus où les termes se recoupent.
- FAISS ne se charge pas sur cette machine à cause d'une DLL bloquée ; le backend NumPy a été utilisé.
- Le modèle d'embeddings est généraliste, pas spécialisé sur le cours.
- Le découpage peut couper une idée malgré le respect des phrases.
- L'index persistant existe, mais la démo le reconstruit encore en mémoire.
- La démo accepte JSON, TXT et MD ; PDF et DOCX ne sont pas pris en charge.
- Ollama n'a pas été lancé avec un vrai modèle ici ; seule la couche HTTP a été testée.
- L'évaluation intégrée utilise les notes d'exemple ; elle est désactivée après un import.
- Aucun reranking ni filtre par cours n'est implémenté.

11. Lancer la démonstration

Depuis PowerShell :

```
cd "D:\ai-study-assistant\ai-study-assistant\modules\semantic_search"
.\.venv\Scripts\python.exe -m streamlit run demo_app.py

# puis ouvrir :
# http://localhost:8501
```

Sans import, la démo charge les notes d'exemple. Un fichier .json, .txt ou .md importé remplace ce corpus. Le slider règle le nombre de résultats. Le premier lancement peut télécharger le modèle d'embeddings.

Pour créer explicitement les embeddings et l'index :

```
.\.venv\Scripts\python.exe -m src.build_index --notes data/notes_exemple.json --output data/index
```

12. Conclusion

Le projet livre un moteur de recherche sémantique fonctionnel, une interface Streamlit, une évaluation comparative, des tests et une documentation de traçabilité. Le pipeline est complet : notes -> passages -> embeddings -> index -> requête -> top-k -> affichage.

La production du code a été pilotée par prompts avec Claude.ai. L'étudiant reste responsable des choix, des vérifications et de l'explication orale. Les résultats présentés sont mesurés ; les limites sont explicitées. C'est cet équilibre entre automatisation et vérification qui rend le travail défendable.

Annexe A — Arborescence du projet

```
modules/semantic_search/
├── search/
│   ├── README.md
│   └── role-card.md
├── data/
│   ├── notes_exemple.json
│   └── evaluation_cases.json
├── src/
│   ├── __init__.py
│   ├── corpus.py
│   ├── module.py
│   ├── build_index.py
│   ├── evaluate.py
│   └── llm.py
├── tests/
│   ├── test_search.py
│   ├── test_llm.py
│   ├── test_index.py
│   └── test_corpus.py
├── demo_app.py
├── requirements.txt
├── prompts.md
├── .gitignore
└── RAPPORT_PROJET_IA_RECHERCHE_SEMANTIQUE.docx
```

Annexe B — Questions probables du professeur

Question	Réponse courte
Pourquoi un modèle d'embeddings plutôt qu'un LLM pour chercher ?	Un embedding compare des vecteurs rapidement et de façon contrôlable. Un LLM génère du texte ; il est plus lent et moins fiable pour classer des passages.
Pourquoi 300 mots et 50 de chevauchement ?	Pour garder une idée entière dans un passage tout en évitant de perdre le contexte à la frontière. La démo fixe ces valeurs pour être reproductible.
À quoi sert FAISS ?	À accélérer la recherche des vecteurs les plus proches. Ici, il est prévu mais bloqué par Windows ; le repli NumPy donne les mêmes résultats sur 6 passages.
Que mesurent Precision@k, Recall@k et MRR ?	Precision@k : part de résultats pertinents dans le top-k. Recall@k : part des notes pertinentes retrouvées. MRR : rang du premier résultat pertinent.
Le sémantique est-il meilleur que les mots-clés ?	Pas systématiquement sur ce corpus de 6 notes. La baseline est forte. Le sémantique a un meilleur MRR dans la démo, mais le jeu est trop petit pour généraliser.
Quelle est la part de l'IA ?	Le code et la documentation ont été produits avec Claude.ai à partir de prompts. L'étudiant a copié les fichiers, choisi les fonctionnalités, lancé les tests et vérifié les résultats.
Où est le LLM dans le projet ?	Le LLM est optionnel. Ollama peut générer une réponse à partir des extraits, mais la recherche fonctionne sans lui. Le modèle central est sentence-transformers.
Pourquoi NumPy au lieu de FAISS ?	FAISS est prévu, mais sa DLL est bloquée par une stratégie Windows sur cette machine. Le code bascule automatiquement sur NumPy.

Annexe C — Déclaration d'usage de l'IA

Déclaration
Ce projet a été réalisé avec Claude.ai. L'utilisateur n'a pas saisi tout le code manuellement : il a piloté l'IA par des prompts, copié les fichiers proposés, choisi les fonctionnalités, puis validé les résultats par des tests et des exécutions réelles. Les prompts, les fichiers produits et les vérifications sont documentés dans ce rapport et dans prompts.md. Aucun résultat n'a été prérempli ou inventé.

Nom de l'étudiant : _____

Date : _____

Signature : _____

Cette déclaration vise la transparence sur la méthode de production. Elle ne remplace pas la capacité à expliquer le code, les choix et les limites du projet.